

Pipeline Development for Strategy 1

Sultan Alhamdan

18/06/2026

Department of Chemical Engineering



Table of Contents

Governing equations	1
State of the Art	2
Proposed Solution	3
Training Approach	4
Why Using Gardner's Model for Controllers	5
Strategy 1 Training Pipeline	6
Penalty Weight λ	7
Speed Benchmarks	8
Work Breakdown Structure (WBS)	9

Governing equations

1

Richard's equation

Water flow through each column obeys one nonlinear PDE. Everything downstream exists to solve or approximate it.

What it is

A mass balance applied per layer of each column. It tracks how *water content* θ changes over time as solution percolates down, balancing storage, diffusion between layers, and gravity drainage.

Why it is challenging

The *diffusivity* D and *conductivity* K both depend on θ itself, so the equation is nonlinear with no closed-form solution. It must be solved numerically, layer by layer, every time step. This is the computational cost PGML targets.

RICHARDS' EQUATION

$$\frac{\partial \theta}{\partial t} = \frac{\partial}{\partial z} \left[D(\theta) \frac{\partial \theta}{\partial z} \right] - \frac{\partial K(\theta)}{\partial z}$$

- **Water content:** how fast moisture changes
- **Diffusion:** spreading between layers
- **Conductivity:** downward drainage

Governing equations

1

Gardner's equation

Where Richards' full physics is too slow for real-time control, the controller swaps in a simpler law for how fluid flows.

What it is

A power law linking *hydraulic conductivity* K to *saturation* S_e . It replaces Richards' nonlinear, saturation-dependent coefficients with one clean expression that is cheap to evaluate.

Why it matters

Gardner stays smooth across the full *saturation range*, so it is fast and stable to solve. This is why this thesis's reference point (Olivares et al., 2025) NMPC uses it internally, and why the surrogate is graded against Gardner physics.

GARDNER'S EQUATION

$$K = K_s \cdot S_e^m$$

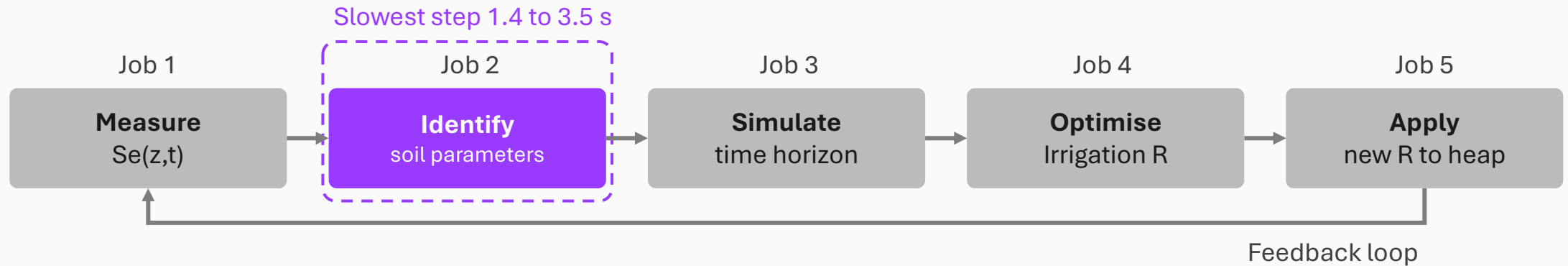
- **Conductivity:** how easily fluid flows through heap
- **Saturated conductivity:** flow rate when heap is fully wet
- **Saturation term:** scales flow down as heap dries out

State of the Art

2

Olivares NMPC

Nonlinear Model Predictive Control (NMPC) regulating irrigation to even out saturation



NMPC solves columns in a **serial manner**. One column takes <5 seconds. At heap scale this becomes computationally expensive at full-heap scale controller.



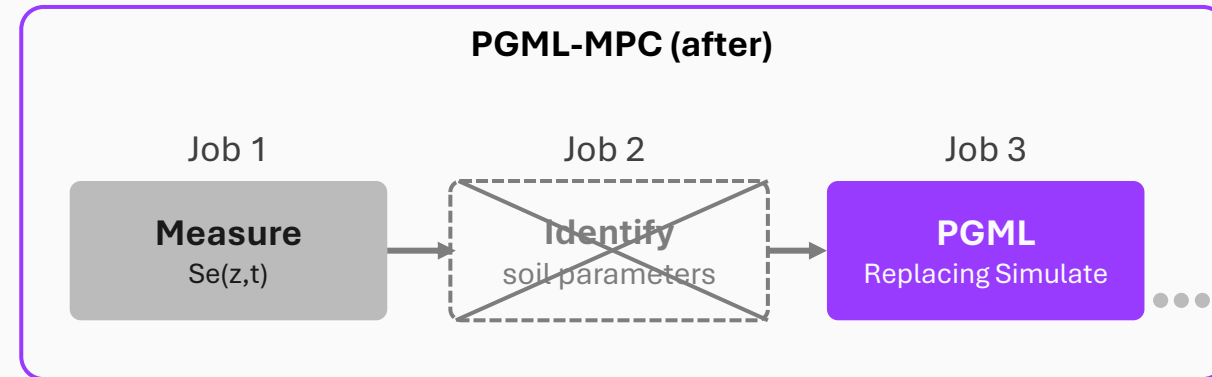
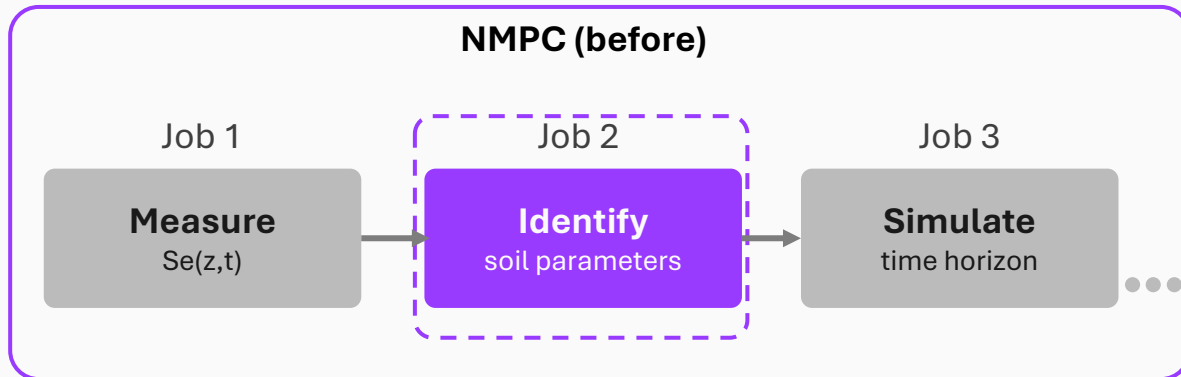
¹ Columns count: 2,000 columns are assumed to form a heap as an initial reference point.

Proposed Solution

3

Physics-Guided Machine Learning (PGML)

Replace NMPC internal model with a neural surrogate eliminating slowest step



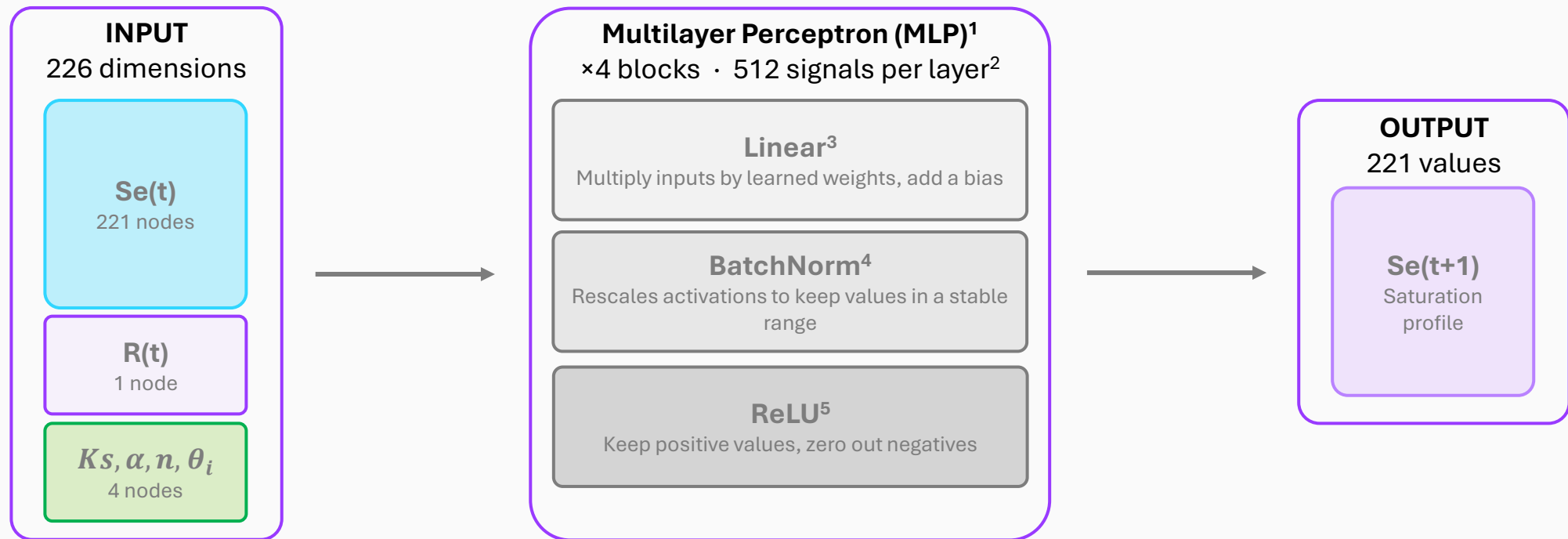
PGML replaces Simulate step (Job 3) with a single sub-second forward pass, and because it trained and learned the soil parameters, it removes the need to Identify (Job 2).

Proposed Solution

3

PGML key components

Feed-forward neural network (NN) that predicts the next profile one hour ahead in a single forward pass



¹ MLP: Stack of neuron layers, simplest possible neural network (NN) architecture.

² Hyperparameters: Repeat three boxes 4 time enabling more capacity to learn patterns. Each layer accepts 512 signals (number), more signals more expressive at training duration cost.

³ Linear: PyTorch function for linear algebra, multiplies every input by a learned weight and adds a bias.

⁴ BatchNorm: PyTorch function for rescaling previous layer outputs to a sensible range.

⁵ ReLU: PyTorch function to replace any negative value with zero, enabling the effect of stacking NN layers.

Training Approach

4

Data only then data and physics

One term fits the data (MSE against **VGM simulator**) with a second term penalises violations of **Gardner physics** using penalty function λ

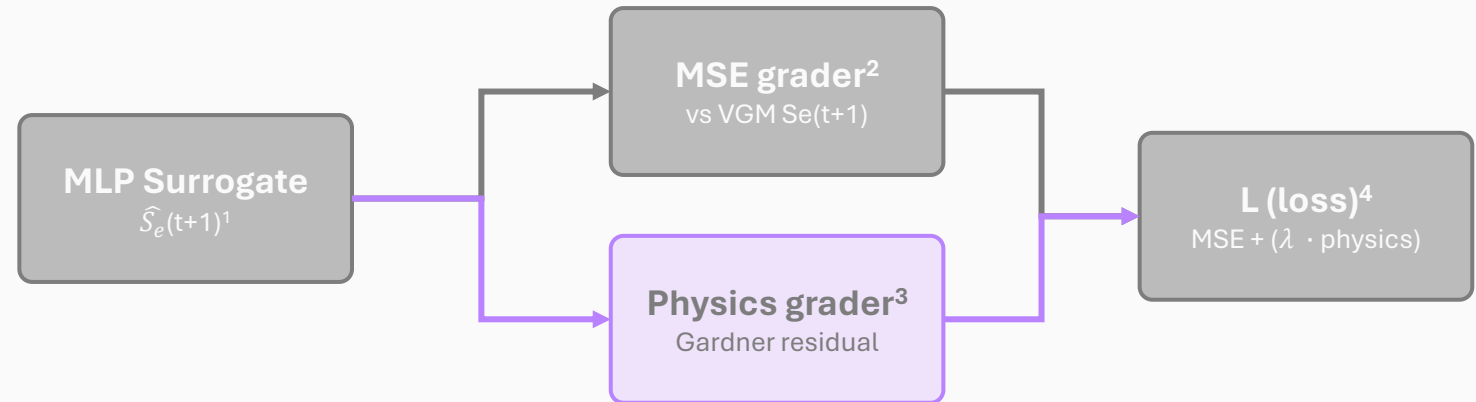
Loss function formula

$$L = \text{MSE}(S_e, \hat{S}_e) + \lambda \cdot R(\hat{S}_e)$$

λ weight for Strategy 1

$$\lambda = 100$$

Training approach



¹ $\hat{S}_e(t+1)$: model's guess at that value, same as S_{e_pred} .

² MSE grader: Mean Squared Error: averages the squared gap between predicted and true saturation across all 221 nodes.

³ Physics grader: Computes the Gardner residual: plugs the prediction back into the Gardner form of Richards' equation and measures how much it fails to satisfy it.

⁴ Loss function: The single number the network is trained to minimise. Combines both graders (data fit + $\lambda \cdot$ physics violation) into one score.

Why Using Gardner's Model for Controllers

5

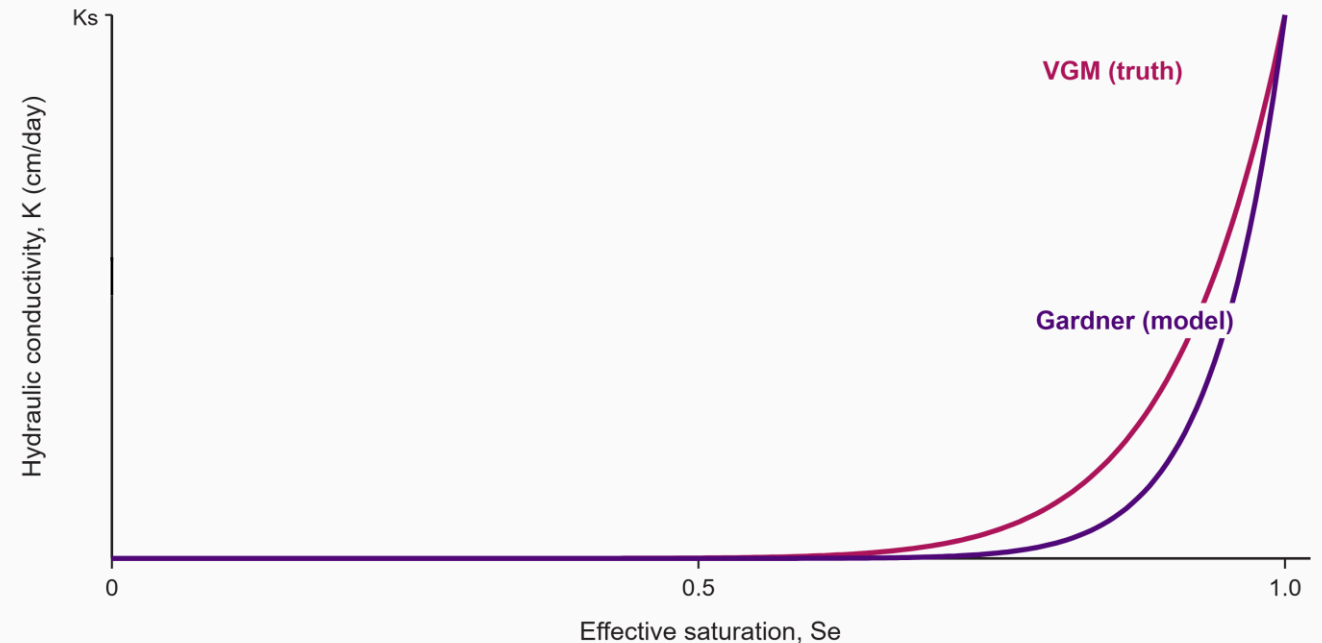
Practical for real-time controllers affected by structural uncertainty

- VGM is the source of truth, hence the driver of generating data for training
- Gardner is deployed inside the controller with a gap from VGM driven by structural uncertainty

Gardner expresses conductivity as a power law. It is what surrogate replaces inside the NMPC, so we grade the surrogate against Gardner physics. It is also numerically stable to train against, where the VGM residual is not.

Gardner's equation

$$K = K_S \cdot S_e^m$$



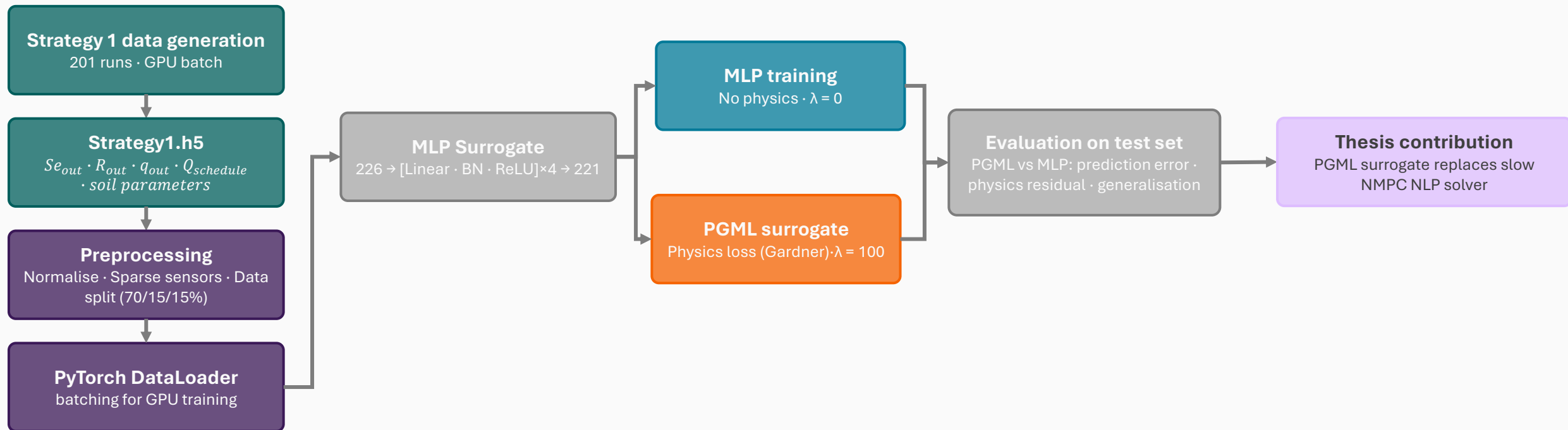
Computed from preliminary findings where VGM $\delta=10.5$ ($n=2.267$), Gardner $m=16.4$ (Olivares fit).

Strategy 1 Training Pipeline

6

Overarching Block Flow Diagram (BFD)

All Matlab files imported from (Olivares et al., 2025) were ported to Python to enable the trailing overarching BFD

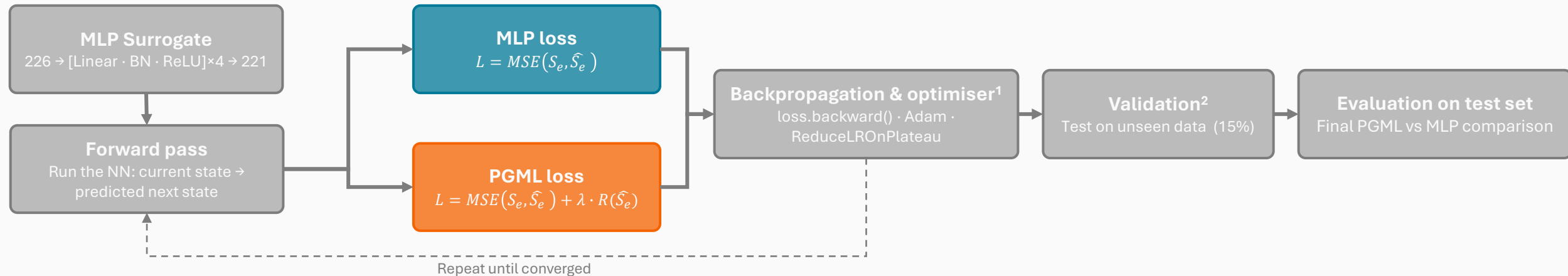


Strategy 1 Training Pipeline

6

Detailed training steps

Trailing BFD shows what happens in between MLP surrogate and Evaluation steps



Validation

- **Job:** prevents overfitting, pick the best moment to halt
- Runs on validation data while training, after each pass
- Checks if the model still improves based on unseen data
- Controls when to stop (early stopping)

Evaluation

- **Job:** PGML final score is the deliverable
- Runs once on the test set, after training is finished
- Reports final metrics: prediction error, physics residual, generalisation
- Compares MLP ($\lambda=0$) vs PGML ($\lambda=100$)

¹ Backpropagation & optimiser: same gradient descent approach by (Olivares et al., 2025) to solve NLP, applied here to tune NN weights instead of solving for irrigation rate R.

² Validation: generated data set is split into 70% for training, 15% for validation and another 15% for evaluation.

Penalty Weight

7

Selecting λ value

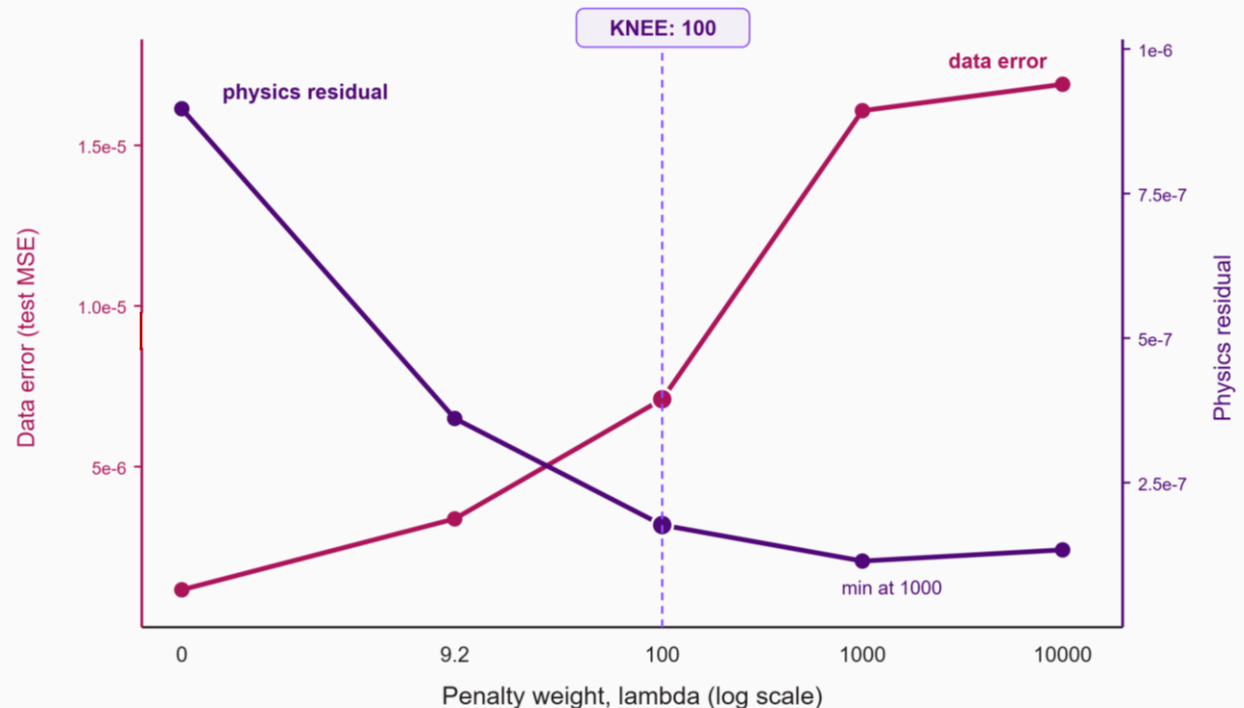
$\lambda=100$ is the best trade-off value for Strategy 1

Initial λ value

started at 9.2, a sensible default but not a justified one. Sweeping values across five values established $\lambda=100$ as the most appropriate fit.

λ value for each strategy

$\lambda=100$ holds for Strategy 1 only. It depends on soil and Gardner fit of this dataset, so the sweep needs to be done for each strategy.



Raising λ trades data accuracy for physics consistency. Gains in physics saturate by $\lambda=100$ beyond which accuracy degrades sharply for insignificant physics benefit.

Speed Benchmarks

8

Eliminating the slow steps

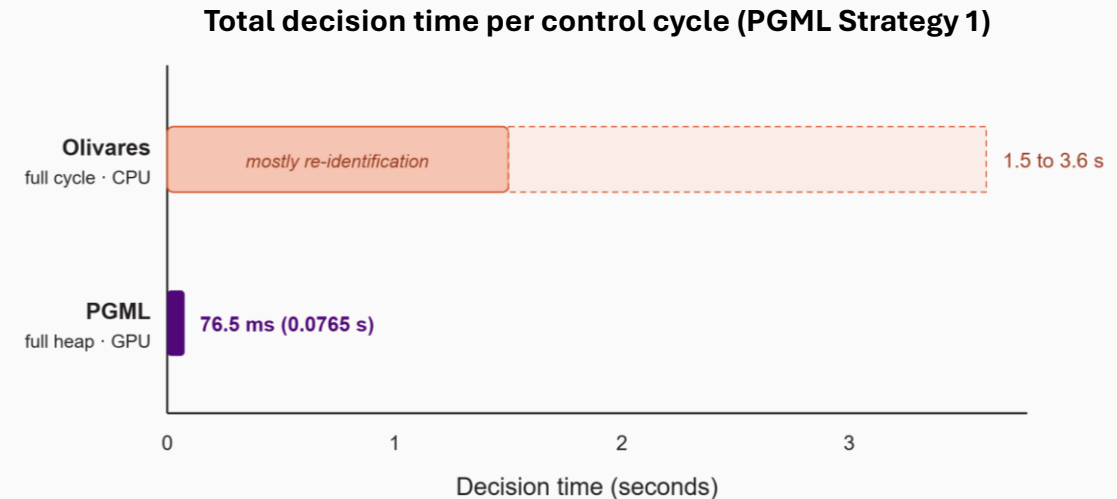
(Olivares et al., 2025) control decision is dominated by re-identification step while PGML removes it, collapsing operational time from seconds into milliseconds

Slowest step

Control cycle proposed by (Olivares et al., 2025) re-identifies the soil parameters (Job 2) before optimising. This step dominates the cost at 1.4 to 3.5 s per decision.

What PGML changes¹

Re-identification step is removed as PGML has learned the soil. Meaning there is nothing to re-fit each control cycle. Remaining decision runs in sub-millisecond time.



Olivares' cycle is dominated by re-identification (up to 3.6 s). PGML eliminates it and runs the full 2000 column heap in 76.5 ms driven by different hardware and eliminated step as the structural change.

PGML control decision vs Olivares reference (Job 4)

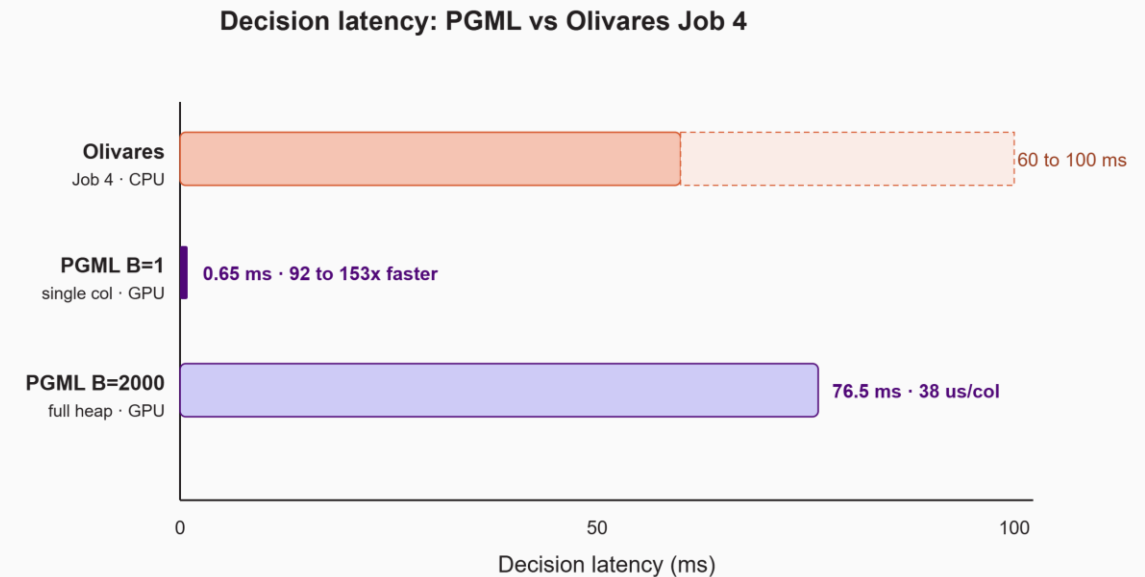
Job 4 is the one step both pipelines share, allowing for like-for-like timing. The bigger win is that PGML eliminates the slow step entirely (identification).

Decision step

The optimisation step (like Olivares' Job 4) sweeps 100 candidate irrigation rates through PGML in one batched pass and picks the best. This is the only job directly comparable to (Olivares et al., 2025).

What the numbers show

A single column decision runs in 0.65 ms and at full heap scale (B=2000) completes in 76.5 ms. The eliminated Identify step is not timed here.



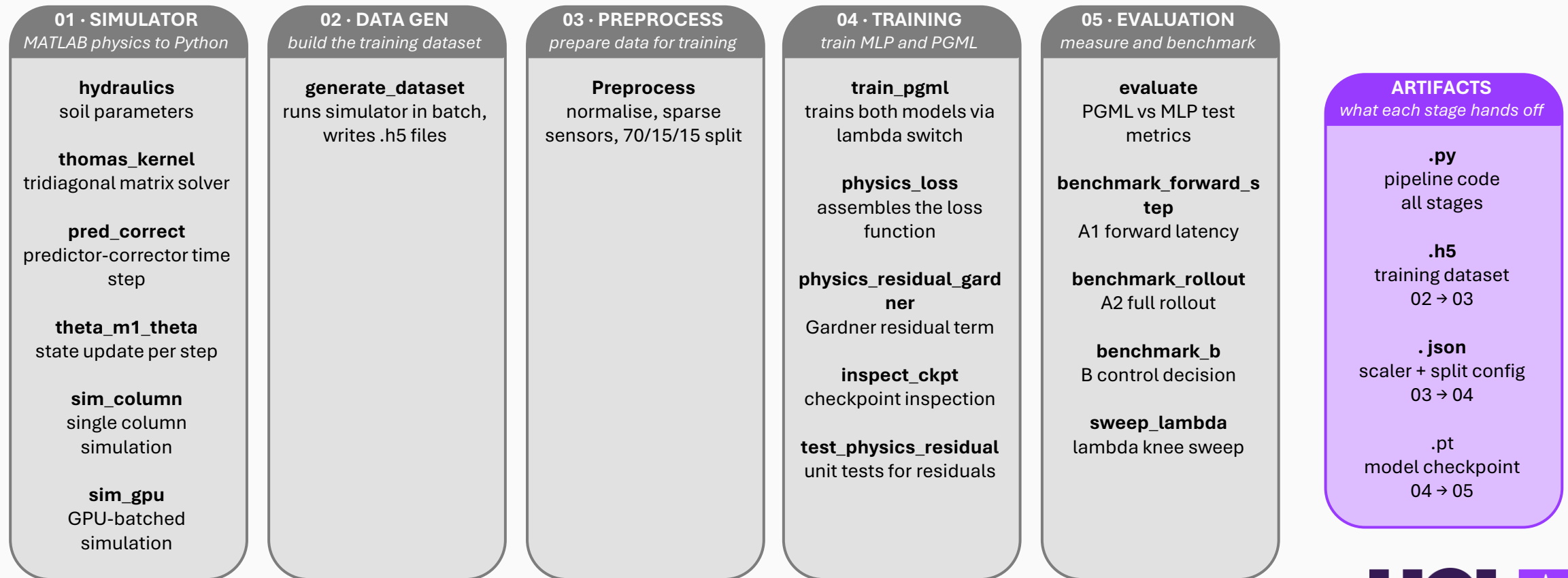
B = batch size (columns per pass). Compares Job 4 (optimisation) only. PGML does the full heap (B=2000) faster than recorded (Olivares et al., 2025) duration for one column.

Work Breakdown Structure

9

Strategy 1 pipeline modules by stage

Each pipeline stage, its modules, and how they connect



Thank you